Assignment 2: Test Automation Implementation

Risk-based automation, quality gates, CI/CD, and metrics

Spring 2026
4 April 2026

SUBMITTED TO

Astana IT University
Faculty of Engineering
Software QA and Testing
AQA
Course Instructor

SUBMITTED BY

Aldiyar Seylkhanov, Alexandr Tyulkov

Contents

1. Introduction

This report applies automated testing to the high-risk areas identified in Assignment 1. The implementation is intentionally narrow: automate the most failure-prone backend helpers first, enforce simple quality gates, and record metrics that can be reused in the research paper.

2. Automated Test Implementation

2.1. Scope Table

Module / Feature	High-Risk Function	Priority	Notes
Spotify export parsing	Validate export entries and extract track IDs	High	Reject invalid rows before import
Retry helper	Recover from transient rate-limit errors	High	Retry only temporary failures
Range helper	Map UI range to backend cutoff date	High	Must be deterministic for reports
ZIP validation	Reject non-ZIP uploads	High	Prevent malformed input early
API controller contracts	Validate status codes and payloads	High	Planned next iteration
Workflow orchestration	Import and sync flow integration	High	Planned next iteration

Table 1: High-risk scope selected for automation

2.2. Test Cases Table

ID	Module	Description	Input	Expected Result	Type	Notes
TC01	Spotify export	Accept valid entry	All required fields; `ms_played = 60000`	Entry accepted	Positive	Baseline case
TC02	Spotify export	Reject missing URI	`spotify_track_uri = null`	Entry rejected	Negative	Schema guard
TC03	Spotify export	Reject short play	`ms_played = 29999`	Entry rejected	Negative	Boundary case
TC04	Spotify export	Extract track ID	`spotify:track:abc123`	`abc123` returned	Positive	Parser check
TC05	Retry helper	Return on success	Resolved promise	No retry	Positive	Fast path
TC06	Retry helper	Do not retry hard failure	`Error("server error")`	Original error thrown	Negative	Failure routing
TC07	Retry helper	Retry and recover	Two rate-limit errors then success	Success after retries	Positive	Recovery case
TC08	Retry helper	Fail after retry budget	Repeated rate-limit errors	Final error thrown	Negative	Budget exhausted
TC09	Range helper	Support `all`	`all`	No cutoff date	Positive	Open range
TC10	Range helper	Map `30d`	Fixed system time; `30d`	Date 30 days earlier	Positive	Deterministic date
TC11	ZIP validation	Detect ZIP signature	`50 4B 03 04`	`true`	Positive	Magic bytes
TC12	ZIP validation	Reject non-ZIP bytes	`7B 22, [], 50 00`	`false`	Negative	Input validation

Table 2: Critical automated test cases

2.3. Script Implementation Table

Script ID	Module	Framework	Location	Status	Comments
S01	Spotify export	Vitest	`apps/backend/src/library/spotify-export.spec.ts`	Complete	Validation and parser coverage
S02	Retry helper	Vitest	`apps/backend/src/library/retry.spec.ts`	Complete	Uses fake timers
S03	Range helper	Vitest	`apps/backend/src/library/range.spec.ts`	Complete	Date mapping checks
S04	ZIP validation	Vitest	`apps/backend/src/library/zip.spec.ts`	Complete	Binary signature checks

Table 3: Automation script tracking

2.4. Version Control Table

Commit ID	Date	Module	Description	Author
`5f4ac20`	2026-03-16	Backend baseline	Created backend package and helper modules	A. Sehl
`2d4a690`	2026-04-04	Backend tests	Added Vitest specs for export, retry, range, and ZIP	A. Sehl
`9c3f879`	2026-04-04	Library refactor	Renamed `src/lib` to `src/library` and updated imports	A. Sehl

Table 4: Version control tracking

2.5. Evidence Table

Evidence ID	Module	Type	Description	File Location
E01	Spotify export	Code	Validation and URI parsing tests	`apps/backend/src/library/spotify-export.spec.ts`
E02	Retry helper	Code	Async retry and back-off tests	`apps/backend/src/library/retry.spec.ts`
E03	Range helper	Code	Date-range mapping tests	`apps/backend/src/library/range.spec.ts`
E04	ZIP validation	Code	ZIP signature checks	`apps/backend/src/library/zip.spec.ts`
E05	Test config	Config	Backend test runner configuration	`apps/backend/vite.config.ts`
E06	CI pipeline	Config	Workflow for check, test, and build	`.github/workflows/ci.yml`

Table 5: Evidence for reproducibility

3. Quality Gate Definition & Integration

3.1. Quality Gate Table

QG ID	Metric	Threshold	Importance	Notes
QG01	Coverage of critical modules	≥ 80%	High	Use coverage report once enabled
QG02	Critical defects	0 allowed on trunk	High	Any failed critical test blocks merge
QG03	Execution time	≤ 5 min total	Medium	Current suite is far below this
QG04	Regression success	100% on critical paths	High	All automated critical tests must pass
QG05	Linting and static checks	Zero major violations	Medium	Enforced through `vp check`

Table 6: Defined quality gates

3.2. CI/CD Pipeline Table

Step	Description	Tool / Framework	Trigger	Notes
1	Checkout code	GitHub Actions	Push / PR	Start from latest commit
2	Install dependencies	`pnpm` in CI	Automatic	Reproducible environment
3	Run checks	`vp check`	Push / PR	Format, lint, type checks
4	Run tests	`vp run test -r`	Push / PR	Runs workspace tests including backend
5	Run build	`vp run build -r`	Push / PR	Confirms build stability
6	Publish status	GitHub status checks	On failure	Used as alerting signal

Table 7: CI/CD integration overview

3.3. Alerting & Failure Handling Table

Scenario	Alert Type	Recipient	Action	Notes
Critical test failure	Status check	Dev + QA	Fix issue, rerun, then merge	Merge blocked while red
Coverage below threshold	PR review note	Dev team	Add missing tests	Formal enforcement pending
Test timeout	Pipeline failure	Dev team	Inspect async logic and timers	Usually indicates hanging work
CI config error	Workflow failure	DevOps / maintainer	Fix YAML or environment issue	Check failing step logs

Table 8: Alerting and failure handling

4. Metrics Collection

4.1. Coverage Table

Automation Coverage (%) = (Automated high-risk functions / Total high-risk functions) × 100

Module	High-Risk Function	Automated?	Coverage %	Notes
Spotify export parsing	Entry validation and URI parsing	Yes	100%	9 tests
Retry helper	Transient error recovery	Yes	100%	4 tests
Range helper	Date cutoff calculation	Yes	100%	5 tests
ZIP validation	Binary signature check	Yes	100%	3 tests
API controller contracts	HTTP response validation	No	0%	Planned
Workflow orchestration	Import and sync integration	No	0%	Planned

Table 9: Automation coverage per high-risk module

Overall coverage is **67%** because 4 of 6 high-risk functions are automated.

4.2. Execution Time Table

Module	Test Cases	Execution Time per Case	Total Time	Notes
Spotify export	9	0.2 ms	85 ms	Startup dominates total time
Retry helper	4	0.8 ms	91 ms	Fake timers remove wait time
Range helper	5	0.4 ms	85 ms	Uses fixed system time
ZIP validation	3	0.3 ms	83 ms	Simple byte-array checks

Table 10: Execution time tracking

Full backend suite result: **21 tests passed in 95 ms** on 2026-04-04.

4.3. Defects Table

Module	Risk	Expected Defects	Defects Found	Pass/ Fail	Notes
Spotify export	High	1	0	Pass	No defects observed
Retry helper	High	1	0	Pass	Recovery logic behaved as expected
Range helper	High	1	0	Pass	No date calculation issues
ZIP validation	High	1	0	Pass	Signature guard stable
API controller contracts	High	2	0	N/A	Not yet automated
Workflow orchestration	High	2	0	N/A	Not yet automated

Table 11: Defects found versus expected risk

4.4. Test Execution Log Table

Test Case ID	Module	Execution Date/Time	Result	Defects	Time	Notes
TC01-TC04	Spotify export	2026-04-04 15:56	Pass	0	85 ms	Isolated file run
TC05-TC08	Retry helper	2026-04-04 15:56	Pass	0	91 ms	Fake timers used
TC09-TC10	Range helper	2026-04-04 15:56	Pass	0	85 ms	Deterministic clock
TC11-TC12	ZIP validation	2026-04-04 15:56	Pass	0	83 ms	Positive and negative cases
All suite	Backend tests	2026-04-04 15:55	Pass	0	95 ms	21 tests total

Table 12: Detailed execution log

4.5. Metrics Report Summary

- Bar chart: automation coverage per module.
- Line chart: execution time by module.
- Table: defects found versus expected risk.

5. Documentation

5.1. Automation Approach & Tool Selection

Section	Details	Reason
Automation approach	Risk-based and regression-focused. High-risk backend helpers were automated first.	Fast feedback on the most failure-sensitive code
Tool selection	Vitest through the Vite+ workflow. CI uses GitHub Actions.	Matches the TypeScript stack and existing repo tooling
Scope	`spotify-export`, `retry`, `range`, and `zip` helpers	These modules sit on the import and filtering path
Reusability	Tests are co-located, deterministic, and small.	Easy to maintain and rerun

Table 13: Automation strategy summary

5.2. Quality Gate Results Table

QG ID	Metric	Threshold	Observed	Notes
QG01	Coverage	≥ 80%	67% overall	Below target because 2 high-risk areas remain manual
QG02	Critical defects	0	0	Passed for automated scope
QG03	Execution time	≤ 5 min	95 ms	Passed
QG04	Regression success	100%	100%	21 of 21 tests passed
QG05	Static checks	0 major issues	Handled in CI	Observed through `vp check` workflow

Table 14: Observed quality gate results

5.3. CI/CD Overview

The automation is integrated into the repository pipeline. Each push or pull request runs checkout, dependency install, quality checks, tests, and build steps. The pipeline

status acts as the primary alerting mechanism and blocks merges when critical checks fail.

5.4. Initial Results Table

Module	Auto-mated?	Cover-age %	Exec Time	Defects	Pass/ Fail
Spotify export	Yes	100%	85 ms	0	Pass
Retry helper	Yes	100%	91 ms	0	Pass
Range helper	Yes	100%	85 ms	0	Pass
ZIP validation	Yes	100%	83 ms	0	Pass
API controller con-tracts	No	0%	N/A	0	N/A
Workflow orches-tration	No	0%	N/A	0	N/A

Table 15: Initial automation outcomes

5.5. Reproducibility Evidence

The report is reproducible because the scripts, CI configuration, and commit history are version-controlled. The automated scope can be rerun from the repository using the existing Vite+ workflow, and the evidence files listed above point directly to the relevant code and pipeline configuration.

6. Deliverables Checklist

Deliverable	Description	File / Location	Status	Notes
Automated test scripts	Critical backend helper tests with positive and negative cases	`apps/backend/src/library/`	Complete	4 spec files
Updated QA strategy document	Automation approach, quality gates, CI/CD, and metrics	`docs/aqa/assignment2/report.typ`	Complete	This report
Quality gate report	Thresholds and observed results	Section in report	Complete	QG01-QG05
Metrics report	Coverage, execution time, defects, logs	Section in report	Complete	Based on current run data
CI/CD evidence	Pipeline steps and alerting	`.github/workflows/ci.yml`	Complete	Used for continuous verification

Table 16: Submission checklist

7. Conclusion

Assignment 2 establishes a usable automation baseline. The current suite is fast and stable, the main pipeline already enforces test execution, and the recorded metrics are sufficient for the methodology and initial results sections of the research paper. The main gap is coverage of controller and workflow integration layers, which should be the first expansion target in Assignment 3.